Kinematic Residual Flow Matching for Vision-Language-Action Policies

Anonymous CVPR submission

Paper ID *****

Abstract

Vision-language-action (VLA) policies generate actions from current observations, making them brittle for dynamic manipulation where objects move during execution. We propose Kinematic-Residual-VLA*, which decomposes action generation into a kinematic branch and a learned residual branch. The kinematic branch projects estimated object velocity into the robot's action space to account for predictable target displacement; a Flow Matching network then learns only the residual manipulation behavior. We call this* Kinematic Residual Flow Matching. *[REWRITE] The key insight is that object velocity is not an arbitrary conditioning variable; it is a physical factor whose first-order effect on actions can be computed analytically and factored out of the learning problem. [talk about experiments, dataset, backbone and results]*

1. Introduction

Robots that manipulate moving objects must solve two coupled problems: understanding *what* action is appropriate, and predicting *where the target will be* at contact time. Standard VLA policies address the first problem by conditioning on the current image, language instruction, and robot state [5, 11, 25]. This works well in quasi-static scenes, but in tasks such as conveyor-belt picking, catching, or handovers, the observation can already be stale by the time the robot executes the decoded action. Minor latency issues in the VLA can cause task failure, thus making dynamic manipulation even more challenging than static ones.

Current VLA research have attempted to solve the problem of stochasticity in the environment by developing asynchornous inference methods and applying changes to model architecture, that prefer speed over accuracy. A natural response is to provide the policy with predictive information: target velocity, future position estimates, or optical-flow features. However, enriching the input does not simplify the output distribution. We want to seek methods that understand the physical meaning of the object's instrinsics as they try and predict the actions to take so. By simply sending object instrinsics as priviliged input information, we are still not really disentagling its physical meaning within the VLA. Majority of the VLA methods out there focus primiarily on static object scenes, or object scenes with simplistic stochasticity. Sometimes, the stochasticity is inherently generated due the VLA's prediction itself, which demands corrective actions by the next action chunk. The solution to most of these methods lies in transforming the model architectures in one way or the other, or finding innovative methods to conduct higher frequency inference.

A major aspect of our idea is to treat the object instrinsics as a guided residual/disturbance in the flow of the action predicted by the VLA. We want to disenta

The central idea of this paper is to treat object velocity as a *structured nuisance factor* rather than an opaque conditioning token. Over a short action-chunk horizon, object velocity induces a first-order displacement in the desired action that can be projected into the robot's action space via a kinematic map. *Kinematic Residual VLA* removes this projected displacement from the training target, trains a Flow Matching network on the residual $\widetilde{\mathbf{x}}_1$, and adds the displacement back at inference time. The neural network models residual grasping behavior; the analytic branch handles predictable target motion.

Contributions.

- We introduce *Kinematic Residual Flow Matching* (KRFM), a conditional action-generation method that removes projected velocity displacement from the Flow Matching target and restores it via an analytic drift at inference.
- We prove that, under a local additive velocity model, residualization collapses all velocity-conditioned action distributions to a common Wasserstein barycenter and strictly reduces action-target variance (Theorem 1).

2. Related Work

Vision–language–action policies. Large robot policies increasingly use vision-language pretraining and action-token decoding to transfer semantic knowledge into robotic control. RT-1 demonstrated large-scale transformer policies

for real-world robot control [5]. RT-2 framed actions as tokens in a VLA model and showed transfer from web-scale vision-language data to manipulation [25]. OpenVLA provides an open-source VLA trained on broad robot datasets [11]. Kinematic-Residual-VLA is complementary to these systems: it does not require a new backbone, but changes the action expert so that predictable target motion is handled outside the neural residual generator.

Generative action policies. Diffusion Policy models visuomotor control as conditional denoising over action sequences [7]. Flow Matching trains continuous normalizing flows by regressing vector fields along probability paths [12]. Conditional Flow Matching and minibatch optimal-transport variants connect simulation-free vector-field learning with transport paths between source and target distributions [20]. Rectified Flow emphasizes straight transport paths and fast generation [13]. We build on this view by treating action generation as transport from a base action distribution to a demonstration action distribution. The difference is that our target distribution is first expressed in a moving, velocity-disentangled action frame.

Predictive perception for dynamic manipulation. The perception interface can be implemented with modern video and geometric modules. Monocular depth estimation provides metric cues [24]; video segmentation provides target masks [17]; dense point tracking provides temporal correspondences [10]; Lucas–Kanade optical flow and Kalman filtering provide lightweight velocity estimation and uncertainty propagation [9, 14]. Our theory assumes that such a module produces an estimated target velocity and covariance; it does not depend on a specific tracker.

Optimal transport and variability reduction. Wasserstein barycenters define central distributions for probability measures in optimal-transport geometry [1, 16, 22]. Variability reduction with optimal transport uses transport maps to remove nuisance-factor variation by aligning conditional distributions to a common barycenter [8]. The dynamic Benamou–Brenier formulation identifies the squared 2-Wasserstein distance with the minimum kinetic action of a mass-preserving probability flow [2, 3]. We specialize these tools to a VLA-specific nuisance factor: target velocity.

3. Preliminaries: Flow Matching for Action Generation

Vision–language–action models condition on visual observations, language instructions, and robot state to predict continuous actions. Following recent flow-based VLA architectures [4], we model the policy as a Conditional Flow Matching (CFM) generator over action chunks [vague sentence. conditioned on what? Combine with next sentence or improve]

Let the robot context be $\mathbf{c} = (\mathcal{I}, \ell, \mathbf{q})$, where $\mathcal{I}$ denotes the visual observation, ℓ is the language instruction, and $\mathbf{q}$ is proprioception data. Add the object's velocity as a condition as well. The policy $F(\mathbf{c}) = \mathbf{a}_{1:H}$ predicts an H-step horizon action chunk $\mathbf{a}_{1:H} \in \mathbb{R}^{H \times d_a}$, where d_a is the action dimension. For the CFM formulation, we work with the flattened representation $\mathbf{x} \in \mathbb{R}^d$, where $d = Hd_a$. mention x is some reference to a

CFM learns a time-dependent vector field $\mathbf{u}_\theta(\mathbf{x}_s, s, \mathbf{c})$ that transports a base distribution p_0 (typically Gaussian) to the conditional action distribution define the conditional action distribution $p_{data}(.|c)$. Does it evolve with time?. Also mention what s means in context here, and how it differs from H (this is important as we have 2 different lambda parameters based on the timings). Given $\mathbf{x}_0 \sim p_0$, $\mathbf{x}_1 \sim p_{\text{data}}(\cdot \mid \mathbf{c})$, and $s \sim \mathcal{U}[0,1]$, the linear interpolation path is $\mathbf{x}_s = (1-s)\mathbf{x}_0 + s\mathbf{x}_1$, with target vector field $\dot{\mathbf{x}}_s = \mathbf{x}_1 - \mathbf{x}_0$. The CFM training objective is

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}\left[\|\mathbf{u}_\theta(\mathbf{x}_s, s, \mathbf{c}) - (\mathbf{x}_s - \mathbf{x}_0)\|_2^2\right], \quad (1)$$

where the expectation is over $\mathbf{x}_0 \sim p_0$, $\mathbf{x}_1 \sim p_{\text{data}}(\cdot \mid \mathbf{c})$, and $s \sim \mathcal{U}[0,1]$ [12, 19]. At inference, one samples $\mathbf{x}_0 \sim p_0$ and integrates $d\mathbf{x}_s/ds = \mathbf{u}_\theta(\mathbf{x}_s, s, \mathbf{c})$ from $s = 0$ to $s = 1$ to obtain the action chunk.

4. Kinematic Residual Flow Matching

We now present our proposed approach. Section 4.1 defines the dynamic manipulation setting and our core modeling assumption. Sections 4.2–4.3 introduce the kinematic velocity projection and the residual training target. Section 4.5 gives the variance-reduction analysis, and Section 4.4 describes inference.

4.1. Problem Setting

We consider manipulation tasks where the target object may move during execution. In addition to the standard VLA context $\mathbf{c}$, each training demonstration includes an object velocity estimate $\mathbf{v}^o$ and an expert action chunk. The dataset is

$$\mathcal{D} = \left\{\left(\mathbf{c}^{(i)}, \mathbf{v}^{o(i)}, \mathbf{x}_1^{(i)}\right)\right\}_{i=1}^{N},$$

where $\mathbf{x}_1^{(i)}$ is the flattened action chunk from demonstration i. The supervised target is drawn from a velocity-conditioned distribution $p_{\text{data}}(\mathbf{x} \mid \mathbf{c}, \mathbf{v}^o)$.

A standard Flow Matching VLA treats $\mathbf{v}^o$ as part of the conditioning and learns a vector field from p_0 to $p_{\text{data}}(\mathbf{x} \mid$

$\mathbf{c}, \mathbf{v}^o$). This is expressive but requires the network to explain two distinct sources of variation: the residual manipulation behavior (grasp geometry, approach direction, contact strategy) and the predictable displacement from object motion. Because the same grasp can produce different action chunks at different object velocities, this velocity variation inflates the target distribution unnecessarily.

We collect $\mathbf{v}^o$ as explicit side information because it is a *structured explanatory variable*: it accounts for a predictable source of variation in the demonstrated actions. Rather than treating it as an opaque conditioning token, we use it to transform the training target directly [6, 21].

Modeling assumption. Over a short action-chunk horizon, we model the effect of object velocity as an additive displacement in the action space. For a fixed context $\mathbf{c}$:

$$p_{\text{data}}(\mathbf{x} \mid \mathbf{c}, \mathbf{v}^o) \approx \left(\tau_{d_{\mathbf{c}}(\mathbf{v}^o)}\right)_{\#} p_{\text{res}}(\mathbf{x} \mid \mathbf{c}), \qquad \tau_{\mathbf{b}}(\mathbf{x}) = \mathbf{x}+\mathbf{b}, \tag{2}$$

where $p_{\text{res}}(\mathbf{x} \mid \mathbf{c})$ is the residual action distribution in a locally static frame, and $d_{\mathbf{c}}(\mathbf{v}^o) \in \mathbb{R}^d$ is the action-space displacement induced by object velocity. The residual distribution may be multimodal (e.g., multiple grasp modes). The role of this model is not to remove such multimodality, but to prevent the same residual modes from appearing artificially spread across different object velocities.

4.2. Kinematic Velocity Projection

The object velocity $\mathbf{v}^o$ is measured in an object, camera, or world frame, while the Flow Matching target $\mathbf{x}_1$ lives in the flattened action-chunk space. We define a context-dependent projection $\Phi_{\mathbf{c}} : \mathbb{R}^{d_o} \to \mathbb{R}^d$ that maps object velocity into the coordinates of the action chunk. The exact form depends on the action representation.

For Cartesian end-effector actions, $\Phi_{\mathbf{c}}$ can be implemented as a horizon-lifted map that repeats or schedules the desired end-effector compensation across the chunk. For joint-space actions, a local Jacobian model gives

$$\Phi_{\mathbf{c}}(\mathbf{v}^o) = L_H J^{\dagger}(\mathbf{q}) R_{\mathbf{c}} \mathbf{v}^o, \tag{3}$$

where $R_{\mathbf{c}}$ maps the object velocity into the desired end-effector compensation frame, $J^{\dagger}(\mathbf{q})$ is the manipulator Jacobian pseudoinverse, and L_H lifts an instantaneous joint velocity into an H-step action chunk [15, 18]. The projection is local: it is intended to be accurate over the short horizon of an action chunk, not over an entire long-horizon manipulation episode.

We convert this projected velocity into an action-space displacement using a scalar schedule $\lambda(s)$ along the generative flow time:

$$d_{\mathbf{c}}(\mathbf{v}^o) = \Lambda \Phi_{\mathbf{c}}(\mathbf{v}^o), \qquad \Lambda = \int_0^1 \lambda(s)\, ds. \tag{4}$$

The scale factors needed to convert physical velocity into chunk displacement, such as the control period or horizon duration, are absorbed into $\Phi_{\mathbf{c}}$ or Λ. This keeps the notation independent of whether the action chunk is represented as Cartesian deltas, joint deltas, velocities, or normalized action tokens.

4.3. Residual Training Target

For a fixed context $\mathbf{c}$, let $\mathbf{V}^o$ denote the random object velocity in the dataset. We center the kinematic displacement as

$$\bar{d}_{\mathbf{c}} = \mathbb{E}[d_{\mathbf{c}}(\mathbf{V}^o) \mid \mathbf{c}], \qquad \Delta_{\mathbf{c}}(\mathbf{v}^o) = d_{\mathbf{c}}(\mathbf{v}^o) - \bar{d}_{\mathbf{c}}.$$

The residual training target is then

$$\widetilde{\mathbf{x}}_1 = \mathbf{x}_1 - \Delta_{\mathbf{c}}(\mathbf{v}^o) = \mathbf{x}_1 - d_{\mathbf{c}}(\mathbf{v}^o) + \bar{d}_{\mathbf{c}}. \tag{5}$$

The centering term $\bar{d}_{\mathbf{c}}$ is not required to construct a valid residual target: in practice, one may simply train on $\mathbf{x}_1 - d_{\mathbf{c}}(\mathbf{v}^o)$, which already removes the velocity-dependent displacement from the supervision signal. Its role is instead statistical. If the average kinematic displacement is nonzero under context $\mathbf{c}$, subtracting $d_{\mathbf{c}}(\mathbf{v}^o)$ alone shifts the conditional mean of the target distribution:

$$\mathbb{E}[\mathbf{x}_1 - d_{\mathbf{c}}(\mathbf{V}^o) \mid \mathbf{c}] = \mathbb{E}[\mathbf{x}_1 \mid \mathbf{c}] - \mathbb{E}[d_{\mathbf{c}}(\mathbf{V}^o) \mid \mathbf{c}].$$

Adding back $\bar{d}_{\mathbf{c}}$ makes the residualization mean-preserving, since

$$\mathbb{E}[\widetilde{\mathbf{x}}_1 \mid \mathbf{c}] = \mathbb{E}[\mathbf{x}_1 \mid \mathbf{c}].$$

Thus, centering removes the velocity-induced fluctuation $d_{\mathbf{c}}(\mathbf{V}^o) - \bar{d}_{\mathbf{c}}$ while keeping the residual target at the same conditional barycenter as the original action distribution. This is desirable statistically because the learned flow models a lower-variance residual distribution without being forced to compensate for an artificial mean shift. When the velocity-induced displacement is already centered within a task, or when estimating $\bar{d}_{\mathbf{c}}$ is inconvenient, setting $\bar{d}_{\mathbf{c}} = 0$ remains a valid and simple implementation choice.

The residual Flow Matching path uses the same base sample $\mathbf{x}_0 \sim p_0$ as the standard VLA objective, but it connects $\mathbf{x}_0$ to $\widetilde{\mathbf{x}}_1$ rather than to the raw target $\mathbf{x}_1$:

$$\widetilde{\mathbf{x}}_s = (1-s)\mathbf{x}_0 + s\widetilde{\mathbf{x}}_1, \qquad s \sim \mathcal{U}[0,1].$$

The Kinematic Residual Flow Matching loss is

$$\mathcal{L}_{\text{KRFM}}(\theta) = \mathbb{E}\left[\left\|\mathbf{u}_\theta(\widetilde{\mathbf{x}}_s, s, \mathbf{c}) - (\widetilde{\mathbf{x}}_s - \mathbf{x}_0)\right\|_2^2\right]. \tag{6}$$

Compared with direct Flow Matching on $\mathbf{x}_1$, this objective changes the target distribution rather than merely adding another conditioning variable. The network is trained to model the residual action distribution, while the kinematic branch accounts for the predictable object-motion displacement.

4.4. Inference

At inference time, the model samples a residual action in the velocity-disentangled frame. Starting from $\mathbf{x}_0 \sim p_0$, we integrate

$$\frac{d\widetilde{\mathbf{x}}_s}{ds} = \mathbf{u}_\theta(\widetilde{\mathbf{x}}_s, s, \mathbf{c}), \qquad \widetilde{\mathbf{x}}_{s=0} = \mathbf{x}_0,$$

and obtain $\widehat{\widetilde{\mathbf{x}}}_1$ at $s = 1$. The final action chunk is recovered by adding back the centered kinematic displacement:

$$\widehat{\mathbf{x}}_1 = \widehat{\widetilde{\mathbf{x}}}_1 + \Delta_{\mathbf{c}}(\mathbf{v}^o) = \widehat{\widetilde{\mathbf{x}}}_1 + d_{\mathbf{c}}(\mathbf{v}^o) - \bar{d}_{\mathbf{c}}. \tag{7}$$

The result is unflattened into an action chunk and executed in a receding-horizon manner. The neural vector field evolves the residual coordinate, while the kinematic branch places the generated action back into the original coordinates using the observed object velocity.

4.5. Variance Reduction Analysis

We now formalize the variance-reduction effect of kinematic residualization. The analysis is local: it is not intended as a global model of manipulation dynamics, but rather as a short-horizon approximation over one action chunk. In this regime, constant-velocity prediction and local kinematic projection provide a first-order description of how object motion shifts the expert action target.

Assumption 1 (Local residual-plus-kinematic action model)**.** *Fix a context value* $\mathbf{c}$*. The raw expert action target admits the decomposition*

$$\mathbf{X}_1 = \mathbf{R}_{\mathbf{c}} + d_{\mathbf{c}}(\mathbf{V}^o), \tag{8}$$

where $\mathbf{R}_{\mathbf{c}}$ *is a residual action random variable,* $d_{\mathbf{c}}(\mathbf{V}^o)$ *is the projected object-velocity displacement in the action space, both terms have finite second moments, and*

$$\mathbf{R}_{\mathbf{c}} \perp\!\!\!\perp \mathbf{V}^o \mid \mathbf{c}.$$

Assumption 1 states that, conditioned on the context, the raw action target consists of two components: a residual manipulation component and a predictable kinematic displacement induced by object velocity. The residual component may still be complex and multimodal; it may encode different grasps, contact strategies, approach directions, or other task-dependent choices. The assumption only requires that object velocity acts locally as an additive shift of the action chunk after projection through $d_{\mathbf{c}}(\cdot)$. Thus, the same residual manipulation modes should not need to be relearned as separate action modes for every observed object velocity.

For a generic action target $\mathbf{Y}$, let

$$\eta^{\mathbf{Y}}_{\mathbf{v}^o} := \mathcal{L}(\mathbf{Y} \mid \mathbf{V}^o = \mathbf{v}^o, \mathbf{c})$$

denote its velocity-conditioned law. Following Wasserstein barycenters and optimal-transport variability reduction [1, 8, 22], define the velocity-induced OT variability of $\mathbf{Y}$ as

$$\mathcal{V}_{\mathrm{OT}}(\mathbf{Y}; \mathbf{V}^o \mid \mathbf{c}) := \inf_{\nu \in \mathcal{P}_2(\mathbb{R}^d)} \mathbb{E}\left[W_2^2\left(\eta^{\mathbf{Y}}_{\mathbf{V}^o}, \nu\right) \mid \mathbf{c}\right]. \tag{9}$$

This quantity measures how far the conditional action distribution moves as the object velocity changes.

Theorem 1 (Object-velocity residualization reduces target variation)**.** *Under Assumption 1, define the residualized target*

$$\widetilde{\mathbf{X}}_1 = \mathbf{X}_1 - d_{\mathbf{c}}(\mathbf{V}^o) + \bar{d}_{\mathbf{c}}, \qquad \bar{d}_{\mathbf{c}} = \mathbb{E}[d_{\mathbf{c}}(\mathbf{V}^o) \mid \mathbf{c}]. \tag{10}$$

Then all velocity-conditioned residual target laws are identical:

$$\mathcal{L}\left(\widetilde{\mathbf{X}}_1 \mid \mathbf{V}^o = \mathbf{v}^o, \mathbf{c}\right) = \mathcal{L}\left(\mathbf{R}_{\mathbf{c}} + \bar{d}_{\mathbf{c}} \mid \mathbf{c}\right) \quad \textit{for all } \mathbf{v}^o. \tag{11}$$

Consequently,

$$\mathcal{V}_{\mathrm{OT}}(\widetilde{\mathbf{X}}_1; \mathbf{V}^o \mid \mathbf{c}) = 0. \tag{12}$$

In contrast, the raw target satisfies

$$\mathcal{V}_{\mathrm{OT}}(\mathbf{X}_1; \mathbf{V}^o \mid \mathbf{c}) = \mathbb{E}\left[\left\|d_{\mathbf{c}}(\mathbf{V}^o) - \bar{d}_{\mathbf{c}}\right\|_2^2 \mid \mathbf{c}\right]. \tag{13}$$

Moreover, the residualized target covariance satisfies

$$\operatorname{tr}\operatorname{Cov}(\widetilde{\mathbf{X}}_1 \mid \mathbf{c}) = \operatorname{tr}\operatorname{Cov}(\mathbf{X}_1 \mid \mathbf{c}) - \operatorname{tr}\operatorname{Cov}(d_{\mathbf{c}}(\mathbf{V}^o) \mid \mathbf{c}). \tag{14}$$

Thus, whenever the projected object-velocity displacement has nonzero conditional covariance, the residualized target has strictly smaller total variance than the raw action target.

Interpretation. The baseline VLA is trained on raw expert actions $\mathbf{X}_1$, whose variation reflects both residual manipulation choices and object-motion compensation. Under Assumption 1, object velocity creates translated copies of the same residual action distribution. Kinematic residualization subtracts the velocity-specific displacement $d_{\mathbf{c}}(\mathbf{V}^o)$ and adds back its conditional mean $\bar{d}_{\mathbf{c}}$, thereby aligning these translated distributions at a common center. The residualized target therefore removes variation that is explained by first-order object motion, while preserving the residual manipulation distribution that the policy must still learn. For a Flow Matching VLA, this means that the terminal action distribution used for training has less velocity-induced spread; the learned generative velocity field no longer needs to model repeated, velocity-shifted copies of the same manipulation modes.

Proof. Let

$$\mathbf{D_c} := d_{\mathbf{c}}(\mathbf{V}^o).$$

By Assumption 1,

$$\mathbf{X}_1 = \mathbf{R_c} + \mathbf{D_c}.$$

Substituting this expression into Eq. equation 10 gives

$$\widetilde{\mathbf{X}}_1 = \mathbf{R_c} + \mathbf{D_c} - \mathbf{D_c} + \bar{d}_{\mathbf{c}} = \mathbf{R_c} + \bar{d}_{\mathbf{c}}.$$

Since $\mathbf{R_c} \perp\!\!\!\perp \mathbf{V}^o \mid \mathbf{c}$, the conditional law of $\mathbf{R_c}$ does not depend on the realized velocity value $\mathbf{v}^o$. Hence, for every $\mathbf{v}^o$,

$$\mathcal{L}\left(\widetilde{\mathbf{X}}_1 \mid \mathbf{V}^o = \mathbf{v}^o, \mathbf{c}\right) = \mathcal{L}\left(\mathbf{R_c} + \bar{d}_{\mathbf{c}} \mid \mathbf{c}\right),$$

which proves Eq. equation 11. Since all velocity-conditioned residual laws are identical, choosing this common law as the barycenter in Eq. equation 9 yields Eq. equation 12.

It remains to compute the OT variability of the raw target. Let

$$\rho_{\mathbf{c}} := \mathcal{L}(\mathbf{R_c} \mid \mathbf{c}).$$

For each velocity value $\mathbf{v}^o$, the raw conditional law is

$$\mathcal{L}(\mathbf{X}_1 \mid \mathbf{V}^o = \mathbf{v}^o, \mathbf{c}) = (T_{d_{\mathbf{c}}(\mathbf{v}^o)})_{\#}\rho_{\mathbf{c}},$$

where $T_{\mathbf{a}}(\mathbf{z}) = \mathbf{z} + \mathbf{a}$ denotes translation by $\mathbf{a}$. Thus, the raw velocity-conditioned laws are translations of the same residual law. Taking

$$\nu^\star = (T_{\bar{d}_{\mathbf{c}}})_{\#}\rho_{\mathbf{c}}$$

gives the upper bound

$$W_2^2\left((T_{d_{\mathbf{c}}(\mathbf{v}^o)})_{\#}\rho_{\mathbf{c}}, (T_{\bar{d}_{\mathbf{c}}})_{\#}\rho_{\mathbf{c}}\right) \le \left\|d_{\mathbf{c}}(\mathbf{v}^o) - \bar{d}_{\mathbf{c}}\right\|_2^2.$$

This bound is tight because the optimal coupling between two translated copies of the same distribution is obtained by applying the corresponding translation. Therefore,

$$\mathcal{V}_{\mathrm{OT}}(\mathbf{X}_1; \mathbf{V}^o \mid \mathbf{c}) \le \mathbb{E}\left[\left\|d_{\mathbf{c}}(\mathbf{V}^o) - \bar{d}_{\mathbf{c}}\right\|_2^2 \mid \mathbf{c}\right].$$

Conversely, for any $\nu \in \mathcal{P}_2(\mathbb{R}^d)$ with mean m_ν, the Wasserstein distance lower bounds the squared distance between means:

$$W_2^2\left((T_{d_{\mathbf{c}}(\mathbf{V}^o)})_{\#}\rho_{\mathbf{c}}, \nu\right) \ge \left\|m_{\rho_{\mathbf{c}}} + d_{\mathbf{c}}(\mathbf{V}^o) - m_\nu\right\|_2^2,$$

where $m_{\rho_{\mathbf{c}}}$ is the mean of $\rho_{\mathbf{c}}$. Minimizing the right-hand side over m_ν yields $m_\nu = m_{\rho_{\mathbf{c}}} + \bar{d}_{\mathbf{c}}$ and gives the lower bound

$$\mathbb{E}\left[\left\|d_{\mathbf{c}}(\mathbf{V}^o) - \bar{d}_{\mathbf{c}}\right\|_2^2 \mid \mathbf{c}\right].$$

The matching upper and lower bounds prove Eq. equation 13.

Finally, using the additive decomposition and conditional independence,

$$\mathrm{Cov}(\mathbf{X}_1 \mid \mathbf{c}) = \mathrm{Cov}(\mathbf{R_c} \mid \mathbf{c}) + \mathrm{Cov}(d_{\mathbf{c}}(\mathbf{V}^o) \mid \mathbf{c}).$$

On the other hand,

$$\widetilde{\mathbf{X}}_1 = \mathbf{R_c} + \bar{d}_{\mathbf{c}},$$

and $\bar{d}_{\mathbf{c}}$ is constant conditioned on $\mathbf{c}$, so

$$\mathrm{Cov}(\widetilde{\mathbf{X}}_1 \mid \mathbf{c}) = \mathrm{Cov}(\mathbf{R_c} \mid \mathbf{c}).$$

Subtracting these two identities and taking traces gives Eq. equation 14. Since covariance matrices are positive semidefinite, a nonzero $\mathrm{Cov}(d_{\mathbf{c}}(\mathbf{V}^o) \mid \mathbf{c})$ has positive trace, yielding strict reduction in total variance. □

5. Experiments

We study Kinematic Residual Flow Matching (KRFM) within the DynamicVLA framework [23]. The experiments address three questions: does residualizing the action target improve dynamic manipulation; does it help under changes in object motion; and can any benefit be attributed to the kinematic decomposition rather than additional velocity information or training? We distinguish the released DynamicVLA checkpoint from matched fine-tuned DynamicVLA variants throughout. **[TODO: Draft status: only the locally evaluated released DynamicVLA row is complete. Empty result cells and framed figures await measurements; proposed analyses below are not claims of completed experiments.]**

5.1. Experimental Setup

Benchmark and protocol. Our primary evaluation uses the released Dynamic Object Manipulation (DOM) simulation suite [23]. Table 1 follows Table 1 of the cited paper (arXiv v2), grouping nine dimensions into interaction, perception, and generalization. The suite contains 90 released configurations, with ten configurations per dimension and 20 trials per configuration, for 1,800 trials per complete evaluation. We preserve the configuration files, reset seeds, task termination rules, and continuous inference with latency-aware action streaming. Success means completing the instructed manipulation; proximity to the moving object alone does not count as success.

The completed released-checkpoint evaluation uses Isaac Sim 4.5.0 and Isaac Lab 2.2.1 on two RTX A6000 GPUs, with simulation and inference on separate GPUs. Appendix A records its provenance and provides the fields needed to document subsequent runs. The matched comparisons should use this same execution protocol and hardware allocation.

Training data and implementation. We build our policies on DynamicVLA and use wrist and opposite-view RGB images, language instructions, and robot state. Matched variants share initialization, episode membership, preprocessing, state/action normalization, sampling, optimizer, and checkpoint-selection rules. Neural velocity conditioning and analytic compensation are varied independently. The full-data DOM training family and the earlier tracking-only study are distinct experiments; the latter is not a source of DOM manipulation success rates. **[TODO: Finalize the dataset revision, exclusions, grouped split, training budget, and selected checkpoints in Tables 5–7.]** State/action statistics follow the shared pretrained normalization; velocity statistics and any fitted centering or phase quantities use training episodes only. Validation selects checkpoints without using the official DOM test score. The final report must identify whether velocity comes from simulator state or a causal visual estimator, including target identification, coordinate frame, filtering, and additional sensor access.

Metrics and aggregation. For a set of N trials with manipulation-success indicators y_i and recorded termination times T_i, we report

$$\text{Success} = \frac{100}{N}\sum_{i=1}^{N} y_i, \qquad \text{Time} = \frac{1}{N}\sum_{i=1}^{N} T_i. \tag{15}$$

Time includes successful and failed trials, including timeouts; it is not inference latency. Each dimension has 200 trials, so the pooled overall success rate also equals the mean of the nine dimension scores. Faster termination must be interpreted jointly with success, since failure can end a trial early. The completed baseline is one suite execution with fixed released seeds, not multiple independent training runs. For the matched study, **[TODO: specify training seeds, independent evaluation repeats, and confidence interval procedure, respecting configuration-level grouping and paired resets.]**

5.2. DOM Manipulation Results

Table 1 separates a released-checkpoint reproduction from controlled comparisons of the proposed extension. The released checkpoint achieves 43.28% overall success and an all-trial mean termination time of 8.62 s over 1,800 verified trials. Its dimension scores are recorded without claiming equivalence to the published evaluation.

The central comparison is between matched DynamicVLA and DynamicVLA + KRFM, with and without neural velocity conditioning. Comparing against the released checkpoint alone would confound residualization with additional training. Comparing the two velocity-conditioned variants holds neural side information fixed while changing the action target. Analytic-only KRFM still consumes velocity through its kinematic branch, even though the neural field does not. **[TODO: Insert matched results, overall and per-dimension differences in percentage points, uncertainty, and cases where KRFM does not improve performance.]**

Figure pending
(a) Target speed (m/s) vs. success (%)
Mark training range and trial counts

Figure pending
(b) Velocity error (m/s) vs. success (%)
Oracle and causal-estimator conditions

Figure 1. Motion sensitivity (plots pending). Use the same method colors in both panels. The speed sweep compares matched task configurations; the error sweep uses a fixed evaluation distribution. Add uncertainty with its resampling unit and report delay/bias conditions separately from random noise.

5.3. Motion Generalization and Velocity Robustness

We separate performance on DOM's released motion-generalization configurations from a controlled speed study. The latter uses fixed task configurations and matched resets across methods while varying target speed. A separate speed-restricted training study is needed to establish generalization to speeds unseen during training: fine-tuning a checkpoint pretrained on full DOM does not by itself establish that exclusion. Table 2 reserves results for this study, and Figure 1 specifies the associated sweeps. **[TODO: Freeze train/validation/test speed intervals, episode counts, split hash, initialization without held-out-speed exposure, and trials per speed bin.]**

For velocity robustness, compare an explicitly labelled simulator-state oracle with a causal estimated-velocity system. Apply controlled noise, bias, and delay to the same velocity stream used by both the neural and analytic branches. Keep the baseline observations and execution schedule fixed. Include zero velocity and incorrect-direction controls to expose dependence on compensation quality. These are controlled perturbation tests, not measurements of real sensor error. **[TODO: Specify the estimator, units, perturbation distributions, delay grid, missing-track behavior, and associated success differences.]**

5.4. Ablating the Kinematic Decomposition

Table 3 isolates neural velocity input, target residualization, and the correction schedule. Constant compensation and causal phase gating use separate trained variants. Phase gating may depend only on information available at the decision time. To test train–inference consistency, disable the analytic reconstruction in a trained KRFM model; report

Table 1. DOM manipulation comparison, following the layout and baseline roster of Table 1 in DynamicVLA [23] (arXiv v2). The nine dimension columns and overall Success are percentages; Time is seconds averaged over all trials. The reference block is reserved for cited published results; the lower block is reserved for our evaluations. Only the released DynamicVLA checkpoint evaluated locally is populated. Blank cells are pending, not zero. No best-method highlighting is applied before comparisons exist.

	Interaction			Perception			Generalization			Average	
Method	CR	DA	LS	VU	SR	MP	VG	MG	DR	Success ↑	Time ↓
Published reference methods (values to be transcribed with attribution)											
Diffusion Policy											
ACT											
OpenVLA-OFT											
$\pi_{0.5}$											
RTC											
SmolVLA											
GR00T-N1.5											
VLA-Adapter-Pro											
VLASH											
DynamicVLA (published)											
Local evaluation under the released DOM protocol											
DynamicVLA (released)	66.00	27.50	31.00	40.00	39.50	34.50	62.50	62.50	26.00	43.28	8.62
DynamicVLA (matched FT)											
DynamicVLA + velocity											
DynamicVLA + KRFM											
DynamicVLA + KRFM + velocity											

CR: closed-loop reactivity; DA: dynamic adaptation; LS: long-horizon sequencing; VU: visual understanding; SR: spatial reasoning; MP: motion perception; VG: visual generalization; MG: motion generalization; DR: disturbance robustness. FT denotes matched fine-tuning. The perception column SR is distinct from overall Success. Velocity source for matched rows: **[to specify]**; simulator-state velocity must be identified as privileged.

Table 2. Controlled speed generalization. Each entry is manipulation success (%). Define the speed intervals and episode counts before filling results. Static denotes a near-zero-speed control; IID and OOD refer to the separately specified training distribution. All cells are pending.

DynamicVLA variant	Static	IID	Slow OOD	Fast OOD
Matched FT				
+ velocity				
+ KRFM				
+ KRFM + velocity				

this as an inference intervention, not as a retrained baseline. **[TODO: Specify the phase rule, centering estimator, physical-to-normalized action map, compensation scale, and the velocity source shared by each comparison.]**

Target-distribution diagnostics. To connect the experiments to Theorem 1, compare raw and residual targets on the same held-out examples and in the same units. Define $V_{\text{raw}} = \operatorname{tr} \widehat{\operatorname{Cov}}(\mathbf{x}_1)$ and $V_{\text{res}} = \operatorname{tr} \widehat{\operatorname{Cov}}(\tilde{\mathbf{x}}_1)$ within matched task/phase/context strata. Report their ratio, displacement variance, and the residual–displacement cross-covariance. Pooling unrelated tasks can obscure the conditional statement of the theorem; a variance decrease alone does not prove its independence assumption. Figure 2 pairs these diagnostics with validation error in reconstructed action space, since raw-target and residual-target training losses are not directly comparable. **[TODO: Define strata, minimum samples, covariance estimator, and seed aggregation; insert measured trends without assuming variance reduction or faster convergence.]**

Table 3. KRFM ablations within DynamicVLA. Vel. indicates neural velocity input; all KRFM rows also use velocity analytically. Constant and phase refer to the compensation gate. Success is the full DOM manipulation rate (%); MSE is held-out reconstructed-action MSE in the common normalized action space. The final two rows are additional controls, pending implementation/evaluation.

Variant	Gate	Vel.	Success ↑	MSE ↓
Matched FT	None	No		
+ velocity	None	Yes		
+ KRFM	Constant	No		
+ KRFM + velocity	Constant	Yes		
+ KRFM, phase	Phase	No		
+ KRFM + velocity, phase	Phase	Yes		
No inference reconstruction	Constant	Yes		
Uncentered KRFM	Constant	Yes		

5.5. Runtime and Failure Analysis

Table 4 separates model inference from the additional cost of velocity estimation and analytic compensation. Measure end-to-end latency from observation availability to action availability, including transfers and all added modules. Use the same hardware, precision, observation window, action horizon, solver, and warm-up procedure across methods.

Figure pending
(a) Optimizer updates vs. validation action MSE
Common reconstructed action space

Figure pending
(b) Matched context strata vs. target variance
Raw and residual targets

Figure 2. Learning and target statistics (plots pending). Compare matched training runs at common update counts and show individual seeds or uncertainty. Use identical examples and normalization for the target-variance comparison; report sample counts and the residual–displacement cross term.

Table 4. Runtime on matched hardware (pending). End-to-end (E2E) latency includes velocity estimation and correction; model-only latency excludes them. All times are milliseconds per action chunk, and memory is peak inference GPU memory.

	Model	E2E latency		Memory
DynamicVLA variant	p50	p50	p95	(GiB)
Matched FT				
+ velocity				
+ KRFM				
+ KRFM + velocity				

Report the median and 95th percentile rather than treating asynchronous throughput as latency. **[TODO: Fill timing sample count, synchronization boundaries, warm-up count, peak-memory measurement, and success under added observation delay.]**

Figure 3 reserves paired rollouts for inspecting interception, grasp, and placement. Include both successes and failures under the same task/reset, with elapsed timestamps and observed versus predicted object motion. Categorize failures using a fixed rubric: target selection, late interception, missed grasp, object drop, and placement/timeout. Abrupt acceleration and erroneous velocity are particularly useful tests of the local kinematic approximation. **[TODO: Insert rollout identifiers, annotated frames, failure counts and denominators, and observations supported by those cases.]**

6. Limitations

The local additive model $\boldsymbol{A}_1 = \boldsymbol{A}^{\mathrm{res}} + d_c(\boldsymbol{V})$ is an approximation. It is most appropriate for short-horizon motion where target velocity is nearly constant and the manipulator Jacobian does not change sharply. It can fail when acceleration, rolling dynamics, deformable objects, occlusion-induced identity switches, joint limits, collision constraints, or contact-rich dynamics dominate the action target.

The independence assumption in Assumption 1 is also idealized. High velocity can change the preferred grasp timing, approach direction, and safety margin. In that case, velocity is not purely a nuisance factor. The residual neural field can still model velocity-dependent residual effects if velocity or predicted beliefs are provided as auxiliary context, but the exact zero-variability conclusion should be interpreted as a clean limiting case rather than an unconditional guarantee.

Finally, Kinematic-Residual-VLA depends on velocity estimation. If the perception module is biased, the analytic drift can move the sampled action in the wrong direction. This motivates the oracle-velocity and estimated-velocity comparisons in Section 5, along with robustness tests under velocity noise.

7. Reproducibility Statement

The paper specifies all random variables, action-space transformations, Flow Matching losses, and sampling equations in Sections 3–4. The theoretical assumptions and theorem statements are in Section 4, with proofs alongside the corresponding statements. The empirical protocol, baselines, metrics, and ablations are specified in Section 5. Appendix A records baseline provenance and reserves the dataset, hyperparameter, checkpoint, and compute fields required to complete the empirical report.

8. Conclusion

Kinematic-Residual-VLA reframes dynamic VLA control as action generation in a moving action frame. Instead of asking a neural generator to learn both residual grasping behavior and predictable target motion, the method projects velocity $\boldsymbol{v}$ into action space and injects it as an analytic drift, while Flow Matching learns the residual target $\widetilde{\boldsymbol{A}}_1$. The optimal-transport analysis clarifies why this should help: residualization aligns velocity-conditioned action distributions at a common barycenter, reduces target covariance, and removes the corresponding Benamou–Brenier kinetic action from the learned transport. The resulting paper structure makes the main claim testable: if target velocity is observable and locally projectable into the robot action space, then factoring it out should improve generalization to unseen target speeds.

Author Contributions

If you'd like to, you may include a section for author contributions as is done in many journals. This is optional and at the discretion of the authors.

Acknowledgments

Use unnumbered third level headings for the acknowledgments. All acknowledgments, including those to funding agencies, go at the end of the paper.

Figure pending

Paired DOM rollouts: matched DynamicVLA / DynamicVLA + KRFM + velocity
Columns: observation, approach, interception, grasp, placement or failure
Include one improvement, one shared failure, and one KRFM regression if observed.
Annotate task/reset ID, timestamps, object trajectory, and end-effector trajectory.

Figure 3. Qualitative dynamic manipulation comparison (frames pending). Each pair uses the same released configuration and reset. Display outcomes and timing without inferring success from tracking proximity. Select examples after quantitative evaluation and disclose the selection rule.

References

[1] Martial Agueh and Guillaume Carlier. Barycenters in the wasserstein space. *SIAM Journal on Mathematical Analysis*, 43(2):904–924, 2011. 2, 4

[2] Luigi Ambrosio, Nicola Gigli, and Giuseppe Savaré. *Gradient Flows: In Metric Spaces and in the Space of Probability Measures*. Birkh"auser, 2 edition, 2008. 2

[3] Jean-David Benamou and Yann Brenier. A computational fluid mechanics solution to the monge–kantorovich mass transfer problem. *Numerische Mathematik*, 84(3):375–393, 2000. 2

[4] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. π_0: A vision-language-action flow model for general robot control, 2025. 2

[5] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, Pete Florence, Chuyuan Fu, Montserrat Gonzalez Arenas, Keerthana Gopalakrishnan, Kehang Han, Karol Hausman, Alexander Herzog, Jonathan Hsu, Brian Ichter, Alex Irpan, Nikhil Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Isabel Leal, Lisa Lee, Sergey Levine, Yao Lu, Igor Mordatch, Karl Pertsch, Kanishka Rao, Krista Reymann, Michael Ryoo, Grecia Salazar, Pannag Sanketi, Pierre Sermanet, Jaspiar Singh, Anikait Singh, Radu Soricut, Huong Tran, Vincent Vanhoucke, Quan Vuong, Ayzaan Wahid, Stefan Welker, Paul Wohlhart, Jasmine Wu, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Tianhe Yu, and Brianna Zitkovich. RT-1: Robotics transformer for real-world control at scale. In *Robotics: Science and Systems*, 2023. 1, 2

[6] Dian Chen, Brady Zhou, Vladlen Koltun, and Philipp Krähenbühl. Learning by cheating. In *Proceedings of the Conference on Robot Learning*, pages 66–75, 2020. 3

[7] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems*, 2023. 2

[8] Kai M. Hung, Andrew Lipnick, Ryan Shìjié Dù, Nina Mortensen, and Esteban G. Tabak. Variability reduction with optimal transport. *Manuscript*, 2022. 2, 4

[9] Rudolph E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 1960. 2

[10] Nikita Karaev, Ignacio Rocco, Benjamin Graham, Natalia Neverova, Andrea Vedaldi, and Christian Rupprecht. CoTracker: It is better to track together. In *European Conference on Computer Vision*, 2024. 2

[11] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan P. Foster, Pannag R. Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. OpenVLA: An open-source vision-language-action model. In *Proceedings of the Conference on Robot Learning*, 2024. 1, 2

[12] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023. 2

[13] Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *International Conference on Learning Representations*, 2023. 2

[14] Bruce D. Lucas and Takeo Kanade. An iterative image registration technique with an application to stereo vision. In *Proceedings of the International Joint Confer-*

ence on Artificial Intelligence, pages 674–679, 1981. 2

[15] Yoshihiko Nakamura. *Advanced Robotics: Redundancy and Optimization*. Addison-Wesley, 1991. 3

[16] Gabriel Peyré and Marco Cuturi. Computational optimal transport with applications to data science. *Foundations and Trends in Machine Learning*, 11(5–6): 355–607, 2019. 2

[17] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Raedle, Chloe Rolland, Laura Gustafson, Eric Mintun, Junting Pan, Kalyan Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross Girshick, Piotr Dollár, and Christoph Feichtenhofer. SAM 2: Segment anything in images and videos. *arXiv preprint arXiv:2408.00714*, 2024. 2

[18] Bruno Siciliano, Lorenzo Sciavicco, Luigi Villani, and Giuseppe Oriolo. *Robotics: Modelling, Planning and Control*. Springer, 2009. 3

[19] Alexander Tong, Nikolay Malkin, Guillaume Huguet, Yanlei Zhang, Jarrid Rector-Brooks, Kilian Fatras, Guy Wolf, and Yoshua Bengio. Conditional flow matching: Simulation-free dynamic optimal transport, 2023. 2

[20] Alexander Tong, Kilian Fatras, Nikolay Malkin, Guillaume Huguet, Yanlei Zhang, Jarrid Rector-Brooks, Guy Wolf, and Yoshua Bengio. Improving and generalizing flow-based generative models with minibatch optimal transport. *Transactions on Machine Learning Research*, 2024. 2

[21] Vladimir Vapnik and Akshay Vashist. A new learning paradigm: Learning using privileged information. *Neural Networks*, 22(5–6):544–557, 2009. 3

[22] Cédric Villani. *Optimal Transport: Old and New*. Springer, 2009. 2, 4

[23] Haozhe Xie, Beichen Wen, Jiarui Zheng, Zhaoxi Chen, Fangzhou Hong, Haiwen Diao, and Ziwei Liu. DynamicVLA: A vision-language-action model for dynamic object manipulation, 2026. 5, 7

[24] Lihe Yang, Bingyi Kang, Zilong Huang, Zhen Zhao, Xiaogang Xu, Jiashi Feng, and Hengshuang Zhao. Depth anything V2. In *Advances in Neural Information Processing Systems*, 2024. 2

[25] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jasmine Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, Quan Vuong, Vincent Vanhoucke, Huong Tran, Radu Soricut, Anikait Singh, Jaspiar Singh, Pierre Sermanet, Pannag R. Sanketi, Grecia Salazar, Michael S. Ryoo, Krista Reymann, Kanishka Rao, Karl Pertsch, Igor Mordatch, Henryk Michalewski, Yao Lu, Sergey Levine, Lisa Lee, Tsang-Wei Edward Lee, Irina Leal, Yuheng Kuang, Dmitry Kalashnikov, Ryan Julian, Nikhil J. Joshi, Alex Irpan, Brian Ichter, Jonathan Hsu, Alexander Herzog, Karol Hausman, Keerthana Gopalakrishnan, Cyprien Fu, Pete Florence, Chelsea Finn, Kaushik Dubey, Danny Driess, Tianli Ding, Krzysztof M. Choromanski, Xi Chen, Yevgen Chebotar, Jose Carbajal, Noah Brown, Anthony Brohan, Montserrat Gonzalez Arenas, and Kehang Han. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Proceedings of the 7th Conference on Robot Learning*, pages 2165–2183, 2023. 1, 2

A. Experiment Details and Completion Record

Released-checkpoint evaluation. The filled row in Table 1 comes from the released `hzxie/dynamic-vla-DOM` checkpoint at revision `c9609d94c5940eb42436a31927d7e80ae454279e`, evaluated with upstream source `a27a06d2ca74d0e987a5e552e01013073e93cfd8`. The report verifies 1,800 trials, 20 for each of 90 released configurations. Configuration hashes and reset seeds are retained in the protocol manifest. The simulator and inference run on separate RTX A6000 GPUs using Isaac Sim 4.5.0 and Isaac Lab 2.2.1. This row is an evaluation of released weights, not a locally trained model. Completion time includes failed trials and no tracking-distance success criterion is applied.

Training-family record. Tables 5–7 are fill-in records for the matched DOM study. Populate them from the frozen family and each selected checkpoint, not from historical tracking recipes or profiler throughput. A configured epoch limit is distinct from completed epochs; report both. Additional fine-tuning from full-DOM weights must be disclosed when discussing held-out demonstrations or speed generalization.

Evaluation and analysis record. For each result, retain the checkpoint hash, training seed, configuration/reset identifier, velocity mode, success flag, termination reason, episode time, and infrastructure status. Exclude infrastructure failures only under a declared rerun rule; do not silently remove manipulation failures. Keep successful-only times, if added, separately labelled. **[TODO: Specify repetitions, uncertainty estimator, failed-run handling, and the paired-comparison analysis.]**

For action-space MSE, specify dimensions, masking, normalization, and the reconstruction from the residual frame. Report translation, orientation, and gripper errors separately if their scales make an aggregate ambiguous. For variance diagnostics, retain the exact example membership, context strata, raw/residual targets, displacement,

Table 5. Shared training and inference specification for the matched DynamicVLA study. Numerical settings are intentionally empty until the final run records are selected. Any per-variant deviation must be listed with its reason.

Field	Setting / record to complete
Framework and initial weights	DynamicVLA; **[TODO: checkpoint revision and weight hash]**
Training data	DOM; **[TODO: revision, exclusion manifest, split fingerprint]**
Trainable parameters	**[TODO: full fine-tuning scope and parameter count]**
Visual inputs	Wrist and opposite cameras; **[TODO: resolution, crop/pad, augmentation]**
Temporal observations	**[TODO: history offsets, timestamps, sampling frequency]**
Action representation	**[TODO: translation, rotation, gripper conventions and units]**
Prediction / execution horizon	/
Control period / flow solver steps	/
State/action normalization	Shared pretrained statistics; **[TODO: statistics hash]**
Velocity source and neural input	**[TODO: oracle/estimator, frame, state slots, normalization]**
Analytic compensation	**[TODO: projection, horizon scaling, centering, gate and clipping]**
Optimizer	**[TODO: optimizer, betas, epsilon, weight decay, gradient clipping]**
Learning rate	
Schedule / warm-up	**[TODO: schedule and warm-up updates]**
Effective batch / microbatch / accumulation	/ /
Sampling / loss weighting	**[TODO: frame sampling, padding masks, tail handling]**
Precision / distributed strategy	**[TODO: precision, GPU count, gradient reduction]**
Maximum epochs / optimizer updates	/
Checkpoint selection	Minimum validation reconstructed-action MSE; **[TODO: anchors and frequency]**
Early stopping	**[TODO: minimum epochs, patience, relative improvement threshold]**
Training seeds / independent runs	/
Hardware / training GPU-hours	**[TODO: device and memory]** /
Execution schedule	Continuous inference and latency-aware action streaming
Software / numerical source	**[TODO: package versions, source digest, runtime manifest]**

Table 6. Data accounting. Enter episodes and action frames after exclusions; record the grouping key and split hash. The official simulator suite is reported in trials rather than demonstration frames and is specified in the text. Empty speed intervals must be defined before a controlled speed-generalization study.

Partition	Episodes	Frames	Speed range (m/s)	Membership / provenance
DOM retained release				**[TODO: revision + exclusions]**
Matched training				**[TODO: split hash]**
Matched validation				**[TODO: grouping key]**
Matched held-out test				**[TODO: grouping key]**
Speed-restricted training				**[TODO: new split + initialization]**
Speed-study validation				**[TODO: selection protocol]**
Speed-study IID test				**[TODO: disjoint episodes]**
Speed-study slow / fast OOD				**[TODO: disjoint speed ranges]**

and cross-covariance. For velocity robustness, freeze estimator inputs, noise/bias/delay grids, and random seeds. For runtime, include estimator time, analytic correction time, transfers, policy time, and complete observation-to-action latency.

Figure completion. The framed figures automatically load PDFs when the following files are present under `paper/figures/`: `success_vs_speed.pdf`, `velocity_robustness.pdf`, `validation_action_mse.pdf`, `target_variance.pdf`, and `dom_qualitative.pdf`. Use the same colors, method labels, and uncertainty convention across plots. Replace "pending" captions with measured descriptions after inserting the figures. **[TODO: Before submission, resolve all TODOs and empty cells, remove author-only completion instructions, audit anonymous references/artifacts, and fit the main-text page budget.]**

Table 7. Per-run completion record for the six matched variants. Duplicate rows for independent training seeds. Selected epoch and completed epochs may differ. No blank row implies a completed training run or evaluation.

DynamicVLA variant	Run key	Seed	Completed epochs	Selected epoch	GPU-h	Checkpoint / eval ID
Matched FT	`dom100`					**[TODO: hash / ID]**
+ velocity	`domv100`					**[TODO: hash / ID]**
+ KRFM	`kr100`					**[TODO: hash / ID]**
+ KRFM + velocity	`krv100`					**[TODO: hash / ID]**
+ KRFM, phase	`krp100`					**[TODO: hash / ID]**
+ KRFM + velocity, phase	`krvp100`					**[TODO: hash / ID]**